AutoML

PyCaret vs Auto Gluon vs Auto Keras
Dec 2022

Area	Feature	PyCaret	Auto Gluon	Auto Keras
Data	Tabular	☑	☑	☑
	Text	☑	☑	☑
	Image	☐	☑	☑
Problem	Supervised	☑	☑	☑
	Unsupervised	☑	☐	☐
	Time Series	☑	☑	☑
Preprocessing	Exploratory Data Analysis	☑	☐	☐
	Data Preparation	☑	☐	☐
	Scale and Transform	☑	☐	☐
	Feature Engineering	☑	☐	☐
	Feature Selection	☑	☐	☐
Phases	Train	☑	☑	☑
	Optimize	☑	☑	☑
	Analyze	☑	☑	☑
	Deploy	☑	☐	☐
	Create App/API	☑	☐	☐
	Monitor	☑	☐	☐
Libraries	Sklearn	☑	☐	☐
	XGBoost	☑	☑	☐
	Neural Nets + Neural Architecture Search	☐	☑	☑
Github	stars	6.6k	5.1k	8.7k
	commits	4.2k	1.3k	1.3k
	contributors	95	94	138

PYCARET

pycaret.gitbook.io/docs/

PyCaret Official

Welcome to PyCaret

GET STARTED

- Installation
- Quickstart
- Tutorials
- Modules
- Preprocessing
- Functions
- Release Notes

LEARN PYCARET

- Official Blog
- Videos
- Examples
- Cheat sheet
- FAQs

PyCaret is an open-source, low-code machine learning library in Python that automates machine learning workflows. It is an end-to-end machine learning and model management tool that exponentially speeds up the experiment cycle and makes you more productive.

Compared with the other open-source machine learning libraries, PyCaret is an alternate low-code library that can be used to replace hundreds of lines of code with a few lines only. This makes experiments exponentially fast and efficient. PyCaret is essentially a Python wrapper around several machine learning libraries and frameworks, such as scikit-learn, XGBoost, LightGBM, CatBoost, spaCy, Optuna, Hyperopt, Ray, and a few more.

The design and simplicity of PyCaret are inspired by the emerging role of citizen data scientists, a term first used by Gartner. Citizen Data Scientists are power users who can perform both simple and moderately sophisticated analytical tasks that would previously have required more technical expertise.

Quick Links

Tutorials	Checkout our official notebooks!
Examples	Example notebooks.
Blog	Tutorials and articles by contributors.
API Reference	The detailed API docs of PyCaret
Video Tutorials	Our video tutorial from various events.

Supervised ML | Unsupervise... | Time Se... | Datas...

Classification

In machine learning, classification refers to a predictive modeling problem where the target to be predicted is a *class label*.

- **Quickstart**
- **API Docs**
- **Tutorial**

Regression

In machine learning, regression refers to a predictive modeling problem where the target to be predicted is a *continuous variable*.

- **Quickstart**
- **API Docs**
- **Tutorial**

Supervised ML | Unsupervised ML | Time Series | Datasets

Clustering

Clustering is the task of dividing the population or data points into a number of groups such that data points in the same groups are more similar to other data points in the same group than those in other groups.

- **Quickstart**
- **API Docs**
- **Tutorial**

Anomaly Detection

Anomaly detection is identifying data points in data that don't fit the normal patterns. It can be useful to solve many problems including fraud detection, medical diagnosis, etc.

- **Quickstart**
- **API Docs**
- **Tutorial**

Natural Language Processing

Natural Language Processing is a field in Artificial Intelligence with the ability of a computer to understand, analyze, manipulate, and potentially generate human language.

- **Quickstart**
- **API Docs**
- **Tutorial**

Association Rules Mining

Association rule learning is a rule-based machine learning method for discovering interesting relations between variables in large databases.

- **Quickstart**
- **API Docs**
- **Tutorial**

Supervised ML | Unsupervised ML | Time Series | Datasets

Time Series (beta)

Time series forecasting is the process of analyzing time series data using statistics and modeling to make predictions and inform strategic decision-making

- **Quickstart**
- **API Docs**
- **Tutorial**

Data Preparation | Scale and Trans... | Feature Engine... | Feature Sele... | Other setup param...

Missing Values

Datasets for various reasons may have missing values or empty records, often encoded as blanks or `NaN`. Most of the machine learning algorithms are not capable of dealing with the missing values. Learn More.

Data Types

Each feature in the dataset has an associated data type such as numeric, categorical, or Datetime. PyCaret automatically detects the data type of each feature. Learn More.

One-Hot Encoding

Categorical features in the dataset contain the label values (ordinal or nominal) rather than continuous numbers. Most of the machine learning algorithms are not capable of handling categorical data without encoding. Learn More.

Ordinal Encoding

When the categorical features in the dataset contain variables with intrinsic natural order such as *Low, Medium, and High*, these must be encoded differently than nominal variables (where there is no intrinsic order for e.g. Male or Female). Learn More.

Cardinal Encoding

When categorical features in the dataset contain variables with many levels (also known as high cardinality features), then typical One-Hot Encoding leads to the creation of a very large number of new features. Learn More.

Handle Unknown Levels

When categorical features in the dataset contain unseen variables at the time of predictions, it may cause problems for the trained model as those levels were not present at the time of training. Learn More.

Target Imbalance

When the training dataset has an unequal distribution of target class it can be fixed using the `fix_imbalance` parameter in the setup. Learn More.

Remove Outliers

The `remove_outliers` function in PyCaret allows you to identify and remove outliers from the dataset before training the model. Learn More.

Data Prepar... | Scale and Transform | Feature Engine... | Feature Sele... | Other setup param...

Normalize

Normalization is a technique often applied as part of data preparation for machine learning. The goal of normalization is to rescale the values of numeric columns in the dataset without distorting the differences in the ranges of values. Learn More.

Feature Transform

While normalization rescales the data within new limits to reduce the impact of magnitude in the variance, Feature transformation is a more radical technique. Transformation changes the shape of the distribution. Learn More.

Target Transform

Target Transformation is similar to feature transformation as it will change the shape of the distribution of the target variable instead of the features. Learn More.

Data Prepar... | Scale and Trans... | Feature Engineering | Feature Sele... | Other setup param...

Feature Interaction

It is often seen in machine learning experiments when two features combined through an arithmetic operation become more significant in explaining variances in the data than the same two features separately. Learn More.

Polynomial Features

In machine learning experiments the relationship between the dependent and independent variable is often assumed as linear, however, this is not always the case. Sometimes the relationship between dependent and independent variables is more complex. Learn More.

Group Features

When a dataset contains features that are related to each other in some way, for example, features recorded at some fixed time intervals, then new statistical features such as mean, median, variance, and standard deviation for a group of such features. Learn More.

Bin Numeric Features

Feature binning is a method of turning continuous variables into categorical values using the pre-defined number of bins. It is effective when a continuous feature has too many unique values or few extreme values outside the expected range. Learn More.

Combine Rare Levels

Sometimes a dataset can have a categorical feature (or multiple categorical features) that has a very high number of levels (i.e. high cardinality features). If such feature (or features) are encoded into numeric values, then the resultant matrix is a sparse matrix. Learn More.

Create Clusters

Creating Clusters using the existing features from the data is an unsupervised ML technique to engineer and create new features. Learn More.

Data Prepar... | Scale and Trans... | Feature Engine... | Feature Selection | Other setup param...

Feature Selection

Feature Selection is a process used to select features in the dataset that contributes the most in predicting the target variable. Working with selected features instead of all the features reduces the risk of over-fitting, improves accuracy, and decreases the training time. Learn More.

Remove Multicollinearity

Multicollinearity (also called *collinearity*) is a phenomenon in which one feature variable in the dataset is highly linearly correlated with another feature variable in the same dataset. Learn More.

Principal Component Analysis

Principal Component Analysis (PCA) is an unsupervised technique used in machine learning to reduce the dimensionality of the data. It does so by compressing the feature space. Learn More.

Ignore Low Variance

Sometimes a dataset may have a categorical feature with multiple levels, where the distribution of such levels is skewed and one level may dominate over other levels. Learn More.

Initialize | Train | Optimize | Analyze | Deploy | Others

setup

This function initializes the experiment in PyCaret and prepares the transformation pipeline based on all the parameters passed in the function. The setup function must be called before executing any other function. It only requires two parameters: `data` and `target`. All the other parameters are optional. Learn More.

Initialize | Train | Optimize | Analyze | Deploy | Others

compare_models

This function trains and evaluates the performance of all the models available in the model library using cross-validation. The output of this function is a scoring grid with average cross-validated scores. Learn More.

create_model

This function trains and evaluates the performance of a given model using cross-validation. The output of this function is a scoring grid with cross-validated scores along with mean and standard deviation. Learn More.

Initialize | Train | Optimize | Analyze | Deploy | Others

tune_model

This function tunes the hyperparameters of a given model. The output of this function is a scoring grid with cross-validated scores of the best model. Search spaces are pre-defined with the flexibility to provide your own. The search algorithm can be random, bayesian, and a few others with the ability to scale on large clusters. Learn More.

ensemble_model

This function ensembles a given model. The output of this function is a scoring grid with cross-validated scores of the ensembled model. Two methods `Bagging` or `Boosting` can be used for ensembling. Learn More.

blend_models

This function trains a Soft Voting / Majority Rule classifier for given models in a list. The output of this function is a scoring grid with cross-validated scores of a Voting Classifier or Regressor. Learn More.

stack_models

This function trains a meta-model over given models in a list. The output of this function is a scoring grid with cross-validated scores of a Stacking Classifier or Regressor. Learn More.

optimize_threshold

This function optimizes the probability threshold for a given model. It iterates over performance metrics at different probability thresholds and returns a plot with performance metrics on the y-axis and threshold on the x-axis. Learn More.

calibrate_model

This function calibrates the probability of a given model using isotonic or logistic regression. The output of this function is a scoring grid with cross-validated scores of calibrated classifier. Learn More.

Initialize | Train | Optimize | Analyze | Deploy | Others

plot_model

This function analyzes the performance of a trained model on the hold-out set. It may require re-training the model in certain cases. Learn More.

evaluate_model

This function uses `ipywidgets` to display a basic user interface for analyzing the performance of a trained model. Learn More.

interpret_model

This function analyzes the predictions generated from a trained model. Most plots in this function are implemented based on the SHAP (Shapley Additive exPlanations). Learn More.

dashboard

This function generates the interactive dashboard for a trained model. The dashboard is implemented using the ExplainerDashboard project. Learn More.

deep_check

This function runs a full suite check over a trained model using the deepchecks library. This function is in experimental mode. Learn More.

eda

This function generates automated Exploratory Data Analysis (EDA) using the AutoViz project. Fully interactive and exportable. Learn More.

check_fairness

This function provides fairness-related metrics between different groups in the dataset for a given model. There are many approaches to evaluate fairness but this function uses the approach known as group fairness, which asks: which groups of individuals are at risk for experiencing harm. Learn More.

get_leaderboard

This function returns the leaderboard of all models trained in the current setup. Learn More.

assign_model

This function assigns labels to the training dataset using the trained model. It is only available for unsupervised modules. Learn More.

Initialize | Train | Optimize | Analyze | Deploy | Others

predict_model

This function generates the label using a trained model. When unseen data is not passed, it predicts the label and score on the holdout set. Learn More.

finalize_model

This function refits a given model on the entire dataset. Learn More.

save_model

This function saves the ML pipeline as a pickle file for later use. Learn More.

load_model

This function loads a previously saved pipeline. Learn More.

save_config

This function saves all the global variables to a pickle file, allowing to later resume without rerunning the setup function. Learn More.

load_config

This function loads global variables from a pickle file into Python. Learn More.

deploy_model

This function deploys the entire ML pipeline on the cloud. Learn More.

convert_model

This function transpiles the trained machine learning model's decision function in different programming languages such as Python, C, Java, Go, C#, etc. Learn More.

create_api

This function takes an input model and creates a POST API for inference. It only creates the API and doesn't run it automatically. To run the API, you must run the Python file using `!python`. Learn More.

create_docker

This function creates a Dockerfile and requirements.txt for deploying API. Learn More.

create_app

This function creates a basic gradio app for inference. Learn More.

- Pros
  - very reach: areas, functionality, deployment
  - github community
- Cons
  - sensitive, and not working with, the current version of sklearn > requires a dedicated env
  - still not well prepared for operationalization
  - results may be a bit worse compared to other AutoML frameworks
  - getting the metrics out is a bit tricky (recquires `pull` from the screen printout)
  - aggressive on-screen printouts

AutoGluon

Tabular Prediction
Multimodal Prediction
Time Series Forecasting
Cloud Training and Deployment
Cheat Sheet
Image Prediction (Legacy Version)
Object Detection (Legacy Version)
Text Prediction (Legacy Version)
AutoGluon Predictors
autogluon.features
autogluon.tabular.models

AutoGluon: AutoML for Text, Image, and Tabular Data

doc release v0.6.0 | stable release v0.6.0 | python 3.7 | 3.8 | 3.9 | license Apache-2.0 | downloads/month 101k | Stars 5.1k
Follow @autogluon 517

AutoGluon enables easy-to-use and easy-to-extend AutoML with a focus on automated stack ensembling, deep learning, and real-world applications spanning image, text, and tabular data. Intended for both ML beginners and experts, *AutoGluon* enables you to:

- Quickly prototype deep learning and classical ML solutions for your raw data with a few lines of code.
- Automatically utilize state-of-the-art techniques (where appropriate) without expert knowledge.
- Leverage automatic hyperparameter tuning, model selection/ensembling, architecture search, and data processing.
- Easily improve/tune your bespoke models and data pipelines, or customize *AutoGluon* for your use-case.

NOTE

Example using AutoGluon to train and deploy a high-performance model on a tabular dataset:

```
>>> from autogluon.tabular import TabularDataset, TabularPredictor
>>> train_data = TabularDataset('https://autogluon.s3.amazonaws.com/datasets/Inc/train.csv')
>>> test_data = TabularDataset('https://autogluon.s3.amazonaws.com/datasets/Inc/test.csv')
>>> predictor = TabularPredictor(label='class').fit(train_data=train_data)
>>> predictions = predictor.predict(test_data)
```

AutoGluon can be applied for prediction tasks that involve image and text data. For adopting state-of-the-art deep learning models for multimodal prediction problems, you may try *autogluon.multimodal*:

```
>>> from autogluon.multimodal import MultiModalPredictor
>>> from datasets import load_dataset
>>> train_data = load_dataset("glue", 'mrpc')['train'].to_pandas().drop('idx', axis=1)
>>> test_data = load_dataset("glue", 'mrpc')['validation'].to_pandas().drop('idx', axis=1)
>>> predictor = MultiModalPredictor(label='label').fit(train_data)
>>> predictions = predictor.predict(test_data)
>>> score = predictor.evaluate(test_data)
```

Area	Feature	PyCaret	Auto Gluon	Auto Keras
Data	Tabular	☑	☑	☑
	Text	☑	☑	☑
	Image	☐	☑	☑
Problem	Supervised	☑	☑	☑
	Unsupervised	☑	☐	☐
	Time Series	☑	☑	☑
Preprocessing	Exploratory Data Analysis	☑	☐	☐
	Data Preparation	☑	☐	☐
	Scale and Transform	☑	☐	☐
	Feature Engineering	☑	☐	☐
	Feature Selection	☑	☐	☐
Phases	Train	☑	☑	☑
	Optimize	☑	☑	☑
	Analyze	☑	☑	☑
	Deploy	☑	☐	☐
	Create App/API	☑	☐	☐
	Monitor	☑	☐	☐
Libraries	Sklearn	☑	☐	☐
	XGBoost	☑	☑	☐
	Neural Nets + Neural Architecture Search	☐	☑	☑
Github	stars	6.6k	5.1k	8.7k
	commits	4.2k	1.3k	1.3k
	contributors	95	94	138

- Pros
  - very reach: areas, functionality, deployment
  - very good results
  - github community
  - operationalization - ready
- Cons
  - installation/compilation of specific libraries (e.g. LightGBMX) is still tricky

AutoKeras

autokeras.com

AutoKeras

Search

AutoKeras
Home
Installation
Tutorials
Extensions
Docker
Contributing Guide
Documentation
About

AutoKeras

AutoKeras: An AutoML system based on Keras. It is developed by DATA Lab at Texas A&M University. The goal of AutoKeras is to make machine learning accessible to everyone.

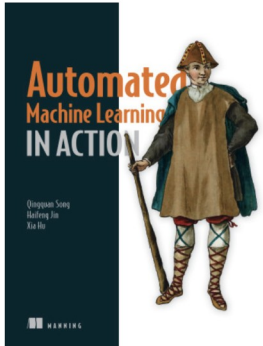
Learning resources

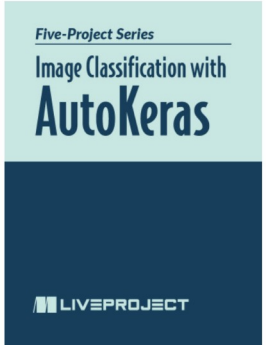
- A short example.

```
import autokeras as ak

clf = ak.ImageClassifier()
clf.fit(x_train, y_train)
results = clf.predict(x_test)
```

- Official website tutorials.
- The book of *Automated Machine Learning in Action*.
- The LiveProjects of *Image Classification with AutoKeras*.





Area	Feature	PyCaret	Auto Gluon	Auto Keras
Data	Tabular	☑	☑	☑
	Text	☑	☑	☑
	Image	☐	☑	☑
Problem	Supervised	☑	☑	☑
	Unsupervised	☑	☐	☐
	Time Series	☑	☑	☑
Preprocessing	Exploratory Data Analysis	☑	☐	☐
	Data Preparation	☑	☐	☐
	Scale and Transform	☑	☐	☐
	Feature Engineering	☑	☐	☐
	Feature Selection	☑	☐	☐
Phases	Train	☑	☑	☑
	Optimize	☑	☑	☑
	Analyze	☑	☑	☑
	Deploy	☑	☐	☐
	Create App/API	☑	☐	☐
	Monitor	☑	☐	☐
Libraries	Sklearn	☑	☐	☐
	XGBoost	☑	☑	☐
	Neural Nets + Neural Architecture Search	☐	☑	☑
Github	stars	6.6k	5.1k	8.7k
	commits	4.2k	1.3k	1.3k
	contributors	95	94	138

- Pros
  - very good for Neural Nets and Neural Architecture Search
  - natural integration with tensorflow and kera
- Cons
  - may be poor on tabular datasets, especially in binary classification with ROC_AUC as measure
  - limitation to Neural Nets